

Design and Implementation of a Two-Wheeled Self-Balancing Robot

Matthew Huynh

Fall 2025

1 Introduction

Two-wheeled self-balancing robots represent a classical example of an unstable, underactuated mechanical system analogous to the inverted pendulum. Maintaining upright balance requires continuous feedback and fast actuation to counteract gravitational torque. As a result, these systems are widely used as educational platforms for control theory, state estimation, and embedded system design.

The objective of this project was to design and implement a self-balancing robot using a model-based Simulink workflow, deployed to an Arduino Mega 2560. Emphasis was placed on practical implementation challenges, including sensor noise, encoder-induced disturbances, actuator saturation, and the interaction between estimation and control dynamics. Rather than focusing solely on theoretical stability, this work highlights real-world limitations encountered during hardware testing.

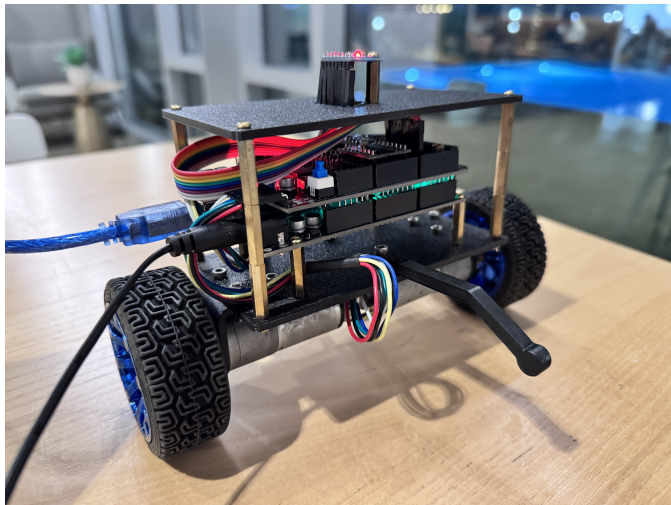


Figure 1: Completed two-wheeled self-balancing robot platform with Arduino Mega, motor driver, and IMU mounted above the wheel axle.

2 System Overview

Figure 1 shows the completed two-wheeled self-balancing robot platform. The mechanical structure consists of a stacked acrylic chassis supporting the Arduino Mega, motor driver board, and inertial measurement unit (IMU) above the wheel axle. This elevated center of mass intentionally increases instability, necessitating active feedback control for balance.

The robot is actuated by two DC motors with magnetic quadrature encoders and driven through a TB6612FNG-based motor driver. All sensing, estimation, and control logic is implemented in Simulink and deployed to the microcontroller using Embedded Coder.

3 Angle Estimation

Accurate estimation of the robot’s tilt angle is critical for stable balance. Raw accelerometer measurements are sensitive to linear acceleration, while gyroscope measurements suffer from drift over time. To address these limitations, a one-dimensional Kalman filter was implemented to fuse accelerometer- and gyroscope-derived information.

The accelerometer-based angle is computed using an `atan2` formulation from the Y and Z acceleration axes, while the gyroscope Y-axis provides angular rate information. The Kalman filter outputs an estimated roll angle, denoted as θ , which is used as the primary feedback signal for the controller.

Figure 2 shows the Simulink implementation of the Kalman filter subsystem.

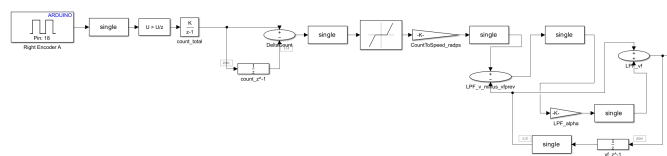


Figure 2: Simulink implementation of the Kalman filter used to estimate roll angle from MPU6050 accelerometer and gyroscope data.

4 Control Architecture

A nested control architecture was adopted, consisting of a fast inner loop for angle stabilization and a slower outer loop for wheel speed regulation. This structure is commonly used for inverted pendulum systems, as it separates the fast unstable dynamics from slower translational motion.

Figure 3 illustrates the top-level Simulink control architecture deployed on the Arduino.

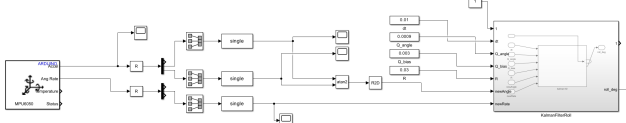


Figure 3: Top-level Simulink control architecture showing the nested angle and speed control loops.

4.1 Inner Loop: Angle Control

The inner loop uses a discrete-time PD controller to regulate the robot's tilt angle. The control error is defined as

$$e_\theta = (\theta_{\text{ref}} + \theta_0 + \theta_{\text{bias}}) - \theta, \quad (1)$$

where θ_0 represents a mechanical offset corresponding to the true upright position and θ_{bias} is provided by the outer speed loop.

To reduce jerky behavior near upright balance, a softening mechanism based on dead-zone and gain shaping was introduced before the PID controller. This reduces control aggressiveness for very small angle errors while preserving full authority for larger deviations.

4.2 Outer Loop: Speed Control

The outer loop estimates wheel speed from encoder pulse counts using a discrete difference,

$$v[k] = \frac{N[k] - N[k-1]}{T_s}, \quad (2)$$

where $N[k]$ is the accumulated encoder count and T_s is the sample period.

A discrete-time PI controller operates on the filtered wheel speed and generates an angle bias term θ_{bias} , which is injected into the inner loop reference. To prevent erroneous bias injection when the robot is lifted or shaken, the speed loop output is gated based on the robot's tilt angle.

Figure 4 shows the encoder pulse accumulation and speed estimation logic.

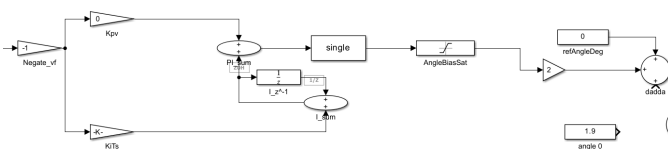


Figure 4: Encoder pulse accumulation, wheel-speed estimation, and low-pass filtering used in the outer speed control loop.

5 Experimental Results

During testing, the robot demonstrated brief periods of stable balance under controlled conditions. The nested control structure reduced slow drift compared to angle-only control; however, sustained balance was limited by encoder noise, actuator nonlinearities, and remaining tuning challenges.

When the robot was lifted off the ground, encoder vibration and mechanical chatter produced false wheel-speed estimates, leading to unintended control action. Gating the speed loop based on tilt angle significantly reduced this effect.

6 Discussion

This project highlights the gap between theoretical control design and real-world implementation. While the inverted pendulum model provides a useful starting point, physical effects such as sensor noise, quantization, backlash, and actuator saturation play a dominant role in observed behavior. Significant effort was required to manage signal conditioning, data types, and discrete-time delays within Simulink.

7 Future Work

Several improvements are planned for future iterations:

- Incorporation of both left and right encoders with averaging
- Improved ground-contact detection using motor current or acceleration magnitude
- State-space or LQR-based control design
- Automated gain tuning with stability guarantees
- Mechanical refinements to reduce backlash and vibration

8 Conclusion

A two-wheeled self-balancing robot was designed and implemented using a model-based Simulink workflow and deployed to an Arduino Mega 2560. Although perfect balance was not achieved, the system successfully integrates state estimation, nested control, and real-time embedded execution. The project provides valuable insight into the practical challenges of implementing unstable control systems on real hardware and establishes a strong foundation for future refinement.